SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA0606– Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	**192521093**
Student Name	**DHARSHITHA.K**

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])          // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

Answer:

M1 – Duplicate-Listing Detector

- Input: n products.
- Basic operation: comparing the title and seller of Product[i] with Product[j].
- The nested loops compare every pair of products. The number of comparisons grows quadratically.
- Time: $\Theta(n^2)$; auxiliary space: O(1).

M2 – API Rate Limiter

- Input: n API requests.
- Basic operation: Audit(Request[j]).
- The inner loop runs 1, 2, 4, 8, ... times as i doubles.
- The total is a geometric series, so the number of operations grows linearly. Time: $\Theta(n)$; space: $O(1)$.

M3 – Product Search

- Input: n products.
- S1 compares products one by one, so its time is $\Theta(n)$.
- S2 compares with the middle item and halves the sorted index each time, so its time is $\Theta(\log n)$.

M4 – Recommendation Engine

- Input: n customer-product records.
- The basic work is recursive processing and merging.
- The current recurrence is $4T(n/2) + n$, giving $\Theta(n^2)$.
- The proposed recurrence is $2T(n/2) + n$, giving $\Theta(n \log n)$.

M5 – Sales Analytics Engine

- Input: n sales records.
- The basic work is sorting and merging.
- For $T(n) = 2T(n/2) + n \log n$, the time complexity is $\Theta(n \log^2 n)$.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

Answer:

M1 – Exact number of comparisons

The outer loop selects each product, and the inner loop compares it with every later product.

$C(n) = (n - 1) + (n - 2) + \ldots + 1$
$C(n) = n(n - 1)/2$
$C(n) = (n^2 - n)/2$

Therefore, time complexity = $\Theta(n^2)$. The algorithm uses only a fixed number of extra variables, so auxiliary space = $O(1)$.

M2 – Exact number of Audit operations
The values of i are 1, 2, 4, 8, ... , 2^k, where $2^k < n \leq 2^{k+1}$.

$C(n) = 1 + 2 + 4 + \ldots + 2^k$
$C(n) = 2^{k+1} - 1$

Since $n \leq 2^{k+1} < 2n$, the tight bound is $\Theta(n)$. The team member's $\Theta(n \log n)$ claim is incorrect: the inner loop does not run n times in every outer-loop iteration.

For n = 1,024, the values are 1 through 512 (powers of two): $C(1{,}024) = 1 + 2 + 4 + \ldots + 512 = 1{,}023$ operations.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

Answer:

Given $T(n) = T(n/2) + 1$, with $T(1) = 1$.

Backward substitution:
$T(n) = T(n/2) + 1$
$= T(n/4) + 1 + 1$
$= T(n/8) + 1 + 1 + 1$
$= T(n/2^k) + k$

At the base case, $n/2^k = 1$.
Therefore, $2^k = n$ and $k = \log_2 n$.

Substitute:
$T(n) = T(1) + \log_2 n$
$T(n) = 1 + \log_2 n$

Exact closed form (for n that is a power of 2): $T(n) = \log_2 n + 1$.
Time complexity: $\Theta(\log n)$.

Space complexity:
• Iterative binary search: O(1), because it uses a fixed number of variables.
• Recursive binary search: O(log n), because the recursive calls use a stack of logarithmic depth.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

Answer:

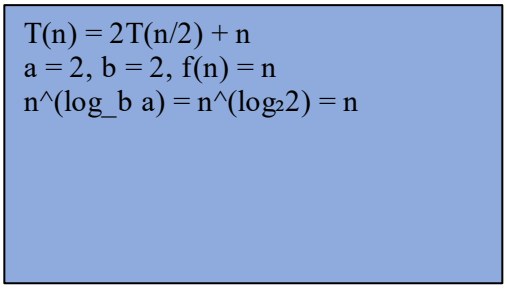
M4 – Current design

$T(n) = 4T(n/2) + n$
$a = 4, b = 2, f(n) = n$
$n^{(\log_b a)} = n^{(\log_2 4)} = n^2$

Since $f(n) = n$ is polynomially smaller than n^2, Master Theorem Case 1 applies. Therefore, $T(n) = \Theta(n^2)$.

If the input doubles, the dominant work changes from n^2 to $(2n)^2 = 4n^2$. It becomes approximately four times as large.

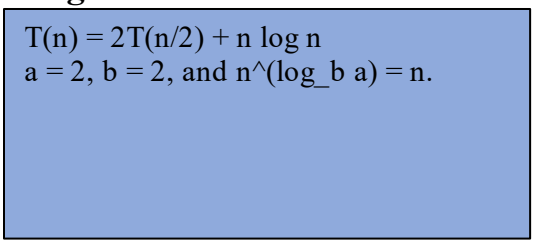
M4 – Proposed redesign



The two terms have the same order, so Master Theorem Case 2 applies.
Therefore, $T(n) = \Theta(n \log n)$.

For doubled input:
$T(2n)$ is proportional to $2n \log_2(2n) = 2n(\log_2 n + 1)$. The growth is approximately twice as large for large n.

M5 – Sales Analytics Engine



Here $f(n) = n \log n = \Theta(n \log^1 n)$. The standard three-case version does not directly cover this logarithmic factor. Using the extended Master Theorem (Case 2):
$T(n) = \Theta(n \log^2 n)$.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

Answer:

S1 – Linear search

• Best case: The product is first in the list; 1 comparison.
• Average successful case: The product is equally likely to be at any position; $(n + 1)/2$ comparisons.
• Worst case: The product is last or absent; n comparisons.

S2 – Binary search (sorted index)

• Best case: The product is at the middle; 1 comparison.

• Average case: $\Theta(\log n)$ comparisons.
• Worst case: The product is at the deepest search level or is absent; $\Theta(\log n)$ comparisons.

Average comparisons for S1 when the product exists with probability p:
If it exists, the average is $(n + 1)/2$ comparisons. If it does not exist, all n items are checked.

$E(n) = p(n + 1)/2 + (1 - p)n$

For $n = 1{,}000{,}000$ and $p = 0.5$:
$E(n) = 0.5(1{,}000{,}001)/2 + 0.5(1{,}000{,}000)$
$E(n) = 250{,}000.25 + 500{,}000$
$E(n) = 750{,}000.25$ comparisons (approximately 750,000).

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that $f(n)$ is not $O(n)$.

Answer:

Given $f(n) = 3n^2 + 10n \log_2 n + 50$.

To prove $f(n) = \Theta(n^2)$, find positive constants c_1, c_2 and n_0 such that:
$c_1 n^2 \leq f(n) \leq c_2 n^2$ for all $n \geq n_0$.

Lower bound:
For $n \geq 1$, all terms are non-negative.
$f(n) = 3n^2 + 10n \log_2 n + 50 \geq 3n^2$.
Choose $c_1 = 3$.

Upper bound:
For $n \geq 16$, $10n \log_2 n + 50 \leq 3n^2$.
Therefore:
$f(n) \leq 3n^2 + 3n^2 = 6n^2$.
Choose $c_2 = 6$.

Choose $n_0 = 16$. Thus, for every $n \geq 16$:
$3n^2 \leq f(n) \leq 6n^2$.

Hence $f(n) = \Theta(n^2)$, with $c_1 = 3$, $c_2 = 6$ and $n_0 = 16$.

To show f(n) is not O(n), divide by n:
$f(n)/n = 3n + 10 \log_2 n + 50/n$.
As n increases, this expression grows without bound. Therefore, no fixed constant c can make $f(n) \leq cn$ for all sufficiently large n. Hence f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Answer:

Given:
n = 1,000,000
Server speed = 10^8 operations/second
Available time = 10 minutes = 600 seconds

Maximum operations in 10 minutes:
$10^8 \times 600 = 60{,}000{,}000{,}000$ operations.

1. M1 – Duplicate-Listing Detector
Operations = $n(n-1)/2$
$= 1{,}000{,}000 \times 999{,}999 / 2$
$= 499{,}999{,}500{,}000$

Time = $499{,}999{,}500{,}000 / 10^8$
$= 4{,}999.995$ seconds ≈ 83.33 minutes.
Result: Fails the 10-minute limit.
Recommendation: Redesign using a hash-based method to group products by title and seller. Expected time can be reduced to $\Theta(n)$, assuming efficient hashing.

2. M2 – API Rate Limiter
For n = 1,000,000, the largest power of two below n is 524,288.
Operations = 1 + 2 + ... + 524,288
= 1,048,575

Time = 1,048,575 / 10^8
≈ 0.0105 seconds.
Result: Passes.
Recommendation: Keep the current algorithm; it has Θ(n) complexity.

3. M3 – Binary Search (S2)
Comparisons ≈ $\log_2(1{,}000{,}000) + 1 \approx 21$.
Time ≈ 21 / 10^8 = 0.00000021 seconds.
Result: Passes.
Recommendation: Use S2 with a sorted product index.

4. M4 – Current design
Dominant operations ≈ n^2 = 1,000,000,000,000.
Time ≈ 1,000,000,000,000 / 10^8
= 10,000 seconds ≈ 166.67 minutes.
Result: Fails.
Recommendation: Replace it with the proposed design to remove replication.

5. M4 – Proposed design
Dominant operations ≈ $n \log_2 n$
≈ 1,000,000 × 19.93
≈ 19,931,569.
Time ≈ 19,931,569 / 10^8
≈ 0.1993 seconds.
Result: Passes.
Recommendation: Use the proposed Θ(n log n) design.

6. M5 – Sales Analytics Engine
Dominant operations ≈ $n(\log_2 n)^2$
≈ 1,000,000 × $(19.93)^2$
≈ 397,300,000.
Time ≈ 397,300,000 / 10^8
≈ 3.973 seconds.
Result: Passes.
Recommendation: Keep the algorithm and monitor its performance.

Summary:

M1: approximately 83.33 minutes – Fail.
M2: approximately 0.0105 seconds – Pass.
M3 (S2): approximately 21 comparisons – Pass.
M4 current: approximately 166.67 minutes – Fail.
M4 proposed: approximately 0.1993 seconds – Pass.
M5: approximately 3.973 seconds – Pass.

Conclusion:
M1 and the current M4 design must be redesigned because they exceed the 10-minute processing window. M2, M3 (S2) and M5 meet the estimated time limit. The proposed M4 design also meets the limit. These running times are estimates based on dominant terms; exact operation counts depend on base cases and implementation details.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I ___**DHARSHITHA.K(192521093)** __ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____